

09/11/2025

Rapport : Projet Programmation Mobile II 2025- 2026



Auteur : Osman Ziylan
ENSEIGNANT : JONATHAN RIGGIO

Table des matières

Introduction	3
Description des technologies utilisés.....	4
React Native & Expo	4
Stockage local avec SQLite.....	4
OCR avec API OCR.space.....	4
Bibliothèques tierces	4
Fonctionnalités	5
Ajout de transactions avec formulaire intelligent	5
Reconnaissance automatique des tickets (OCR).....	6
Validation manuelle des données extraites par OCR	6
Affichage des transactions (Accueil)	7
Tri chronologique des transactions	7
Gestion des Catégories personnalisées.....	8
Initialisation de la base et insertion de “Divers” :	8
Attribution des couleurs :	8
Ajout avec vérification de la limite :	8
Suppression et réaffectation :.....	9
Suivi mensuel et statistiques visuelles.....	9
Choix de la base de données locale (SQLite)	10
Analyse.....	11
Architecture globale	11
Interactions utilisateur : Cas concret de l'ajout de transaction.....	11
Relations entre les couches de l'architecture	12
Vue d'ensemble avec le diagramme de classes principal	12
UX/UI.....	13
ImageButton pour une interface intuitive	13
Ajout d’une barre de recherche (SearchBar).....	14
Design : Une interface inspirée de l’univers des cartes.....	14
Attribution de couleurs par catégorie	14
Héritage visuel – Une signature graphique entre Musio et Budgio	15
Mode jour/nuite automatique	15
Remplacement de l'activité MobileXpo.....	15
Feedback de l’étudiante.....	15

Limitations et développement futur	17
Limitations	17
Connexion Internet	17
Stockage local des données	17
Limitation de l'API OCR gratuite	17
Fiabilité de l'OCR	17
Limite de catégories personnalisées	17
Statistiques limitées	17
Usage mono-utilisateur	17
Gestion des devises	17
Développements futurs	18
Mise en place d'une fonctionnalité de tirelire	18
Amélioration de la reconnaissance OCR	18
Sauvegarde et synchronisation des données	18
Extension des statistiques	18
Catégorisation intelligente	18
Intégration de données économiques en temps réel	18
Gestion multidevise	18
Conclusion	19
Liens YouTube vers les vidéos	20
Vidéo de présentation	20
Vidéo Bonus	20

Rapport du projet Budgio

Introduction

Budgio est une application mobile développée en **React Native** durant l'année académique 2025-2026. Inspirée des **interfaces bancaires modernes**, elle vise à simplifier le suivi des dépenses personnelles, en particulier dans le contexte économique actuel. Accessible à tout utilisateur mené d'un smartphone, **Budgio** facilite la gestion budgétaire grâce à une interface intuitive et des outils automatisés, pensés pour aider chacun à mieux maîtriser ses finances quand le pouvoir d'achat est mis à rude épreuve.

Parmi les **fonctionnalités essentielles**, on trouve :

- **Gestion des transactions** : Ajout, modification et suppression de revenus/dépenses avec catégorisation personnalisée
- **OCR intelligent** : Scan automatique des tickets de caisse via caméra
- **Tableaux de bord visuels** : Graphiques linéaires et circulaires pour suivre l'évolution financière
- **Suivi de budget mensuel** : Définition d'un budget avec alertes en cas de dépassement
- **Stockage local sécurisé** : Base de données SQLite pour la persistance des données
- **Catégorisation avancée** : Jusqu'à 20 catégories par type, entièrement personnalisables

Ce rapport vise à présenter une **vue complète** du projet en abordant les points suivants :

- **Description des technologies utilisées** : Présentation des technologies employées et justification de leur choix.
- **Fonctionnalités** : Description des différentes fonctionnalités de l'application, incluant leur implémentation et leur utilité pour les utilisateurs, ainsi que les choix technologiques et les défis rencontrés.
- **Analyse** : Évaluation des performances de l'application, de son efficacité, de sa stabilité et de sa convivialité, intégrant les retours des utilisateurs et des aspects UX/UI.
- **Limitations** : Mise en lumière des contraintes et des limites rencontrées, ainsi que des aspects à améliorer pour offrir une meilleure expérience utilisateur.
- **Développements futurs** : Recommandations pour les futures améliorations et évolutions de l'application, basées sur les leçons tirées de ce projet et les suggestions des utilisateurs.

Budgio se positionne comme une plateforme innovante qui démocratise la gestion budgétaire personnelle et encourage les bonnes pratiques financières grâce à des outils modernes et accessibles. Ce rapport détaille les aspects techniques, fonctionnels et créatifs qui ont façonné son développement.

Description des technologies utilisés

Budgio repose sur une architecture mobile moderne, pensée pour la performance locale, la simplicité d'intégration et l'autonomie hors ligne. Voici les principaux choix techniques qui ont structuré le projet :

React Native & Expo

Le projet est développé avec **React Native**, associé à **Expo**, pour garantir une compatibilité multiplateforme (iOS et Android) avec une base de code unique. **Expo** facilite l'accès aux fonctionnalités matérielles (caméra, stockage, navigation) tout en réduisant la configuration initiale. Ce choix favorise la productivité, la cohérence du projet et l'accès à des outils intégrés comme les mises à jour OTA¹. Cette implémentation se base sur le tutoriel « Android Studio + Expo Setup Guide (Complete React Native Tutorial) | Monika Szucs BCIT »²

Stockage local avec SQLite

Budgio utilise **SQLite** via Expo pour stocker localement toutes les données critiques : transactions, catégories, budgets. Ce choix garantit un fonctionnement hors ligne, une sécurité renforcée (aucune donnée sur serveur), et des performances optimales pour les opérations fréquentes.

OCR avec API OCR.space

L'analyse automatique des tickets repose sur l'**API OCR.space**, qui extrait les informations clés (montant, date, commerçant) à partir d'une photo. Cette intégration améliore la rapidité de saisie et réduit les erreurs manuelles.

Bibliothèques tierces

Des bibliothèques React enrichissent l'interface :

- **DateTimePicker** : Pour une sélection de date fluide et native
- **React Native Charts** : Pour les visualisations statistiques
- **Expo Router** : pour une navigation par onglets intuitive
- **Expo Vector Icons** : pour une cohérence esthétique à l'aide des icônes

Ces choix technologiques ont permis de construire une base solide, sur laquelle s'appuient les différentes fonctionnalités de **Budgio**. Nous allons à présent détailler ces fonctionnalités, en mettant en lumière leur utilité, leur implémentation technique, ainsi que les défis rencontrés lors du développement.

¹ OTA (Over-The-Air) : Mécanisme de mise à jour qui permet de déployer des modifications de code JavaScript directement sur les appareils des utilisateurs sans passer par les stores officiels

² Lien vers le tutoriel : <https://www.youtube.com/watch?v=NKcuekw4yE4>

Fonctionnalités

Ajout de transactions avec formulaire intelligent

Description : Les utilisateurs peuvent ajouter ou modifier une transaction (revenu ou dépense) via un formulaire complet. Chaque transaction contient un titre, un montant, une date, un mode de paiement et une catégorie. Le formulaire est optimisé pour la saisie mobile avec un formatage intelligent du montant et une navigation fluide.

Défis rencontrés et solutions apportées : Lors de la mise en place du formulaire, des ajustements ont été apportés à la structure des tables dans le code source de la base de données SQLite. Cependant, ces modifications n'étaient pas prises en compte à l'exécution, car la base existante conservait l'ancien schéma. Ce comportement a entraîné des incohérences entre le code et les données persistées. La solution proposée par l'IA a été de modifier le nom du fichier de base dans `storage.ts`³, ce qui a permis de générer une nouvelle base locale avec les propriétés mises à jour :

```
const db = openDatabaseSync("budgio_v1.db");
```



```
const db = openDatabaseSync("budgio_v3.db");
```

D'autres optimisations ont été intégrées pour renforcer la robustesse du formulaire :

- ✓ Le champ montant est filtré pour n'accepter que les chiffres et la virgule, évitant ainsi les saisies erronées :

```
59 const cleanedText = text.replace(/[^d,]/g, '');
```
- ✓ Le formulaire s'adapte dynamiquement au type de transaction (revenu ou dépense), déclenchant automatiquement le rechargement des catégories associées⁴ :

```
41 useEffect(() => {
42   loadCategories();
43   // Réinitialiser le formulaire quand le composant est monté
44   resetForm();
45 }, [isIncome]);
46
47 const loadCategories = async () => {
48   await initDatabase();
49   const cats = await getCategories(isIncome ? 'income' : 'expense');
50   setCategories(cats);
51 };
```
- ✓ Pour garantir une bonne ergonomie mobile, le composant `KeyboardAwareScrollView`⁵ a été utilisé pour éviter les chevauchements avec le clavier
- ✓ Le formulaire est aussi utilisé pour modifier une transaction. L'ID de l'opération est transmis à l'écran d'édition, ce qui permet de charger automatiquement ses données et de les afficher dans les champs pour modification.

Ces choix techniques ont permis de construire un formulaire modulaire, réactif et adapté aux usages mobiles, tout en assurant la cohérence des données et une expérience utilisateur fluide.

³ Ce code se trouve dans le répertoire : « ..\app\utils\storage.ts » L3

⁴ Voir les lignes 41 à 59 et la ligne 59 du script du répertoire : « ..\app\tabs\add-transactions.tsx »

⁵ `KeyboardAvoidingView` est un composant React qui ajuste automatiquement la position de son contenu lorsque le clavier apparaît, voici la documentation : <https://www.npmjs.com/package/react-native-keyboard-avoiding-scroll-view>

Reconnaissance automatique des tickets (OCR)

Description : Cette fonctionnalité permet aux utilisateurs de photographier un ticket de caisse pour en extraire automatiquement les informations grâce à la technologie OCR. Le système analyse l'image et préremplit intelligemment le formulaire de dépense avec le montant, la date, le nom du commerçant et le mode de paiement détectés.

Défis rencontrés et solutions apportées : L'intégration initiale prévoyait l'utilisation de Google ML Kit, mais sa documentation ne proposait que des implémentations en Kotlin et Java, rendant son usage incompatible avec React Native. Une tentative avec @react-native-ml-kit a permis de visualiser le texte, mais pas de l'extraire de manière exploitable. Une seconde piste avec Tesseract OCR a également été testée, mais l'implémentation s'est révélée complexe, lente, et peu adaptée aux contraintes mobiles. Face à ces limitations, la technologie OCR.Space a été retenue pour sa simplicité d'intégration, son efficacité, et son quota mensuel généreux. Elle a permis de stabiliser la fonctionnalité sans configuration native, tout en répondant aux besoins du projet.

Néanmoins, la documentation fournie par OCR.Space manquait de clarté sur certains points techniques. C'est grâce à l'assistance de DeepSeek⁶ que l'implémentation a pu être finalisée avec succès.

Validation manuelle des données extraites par OCR

Description : Lorsque l'utilisateur photographie un ticket de caisse, les données extraites par OCR (montant, date, commerçant, mode de paiement) sont automatiquement injectées dans le formulaire de dépense. Toutefois, avant tout enregistrement, l'utilisateur doit valider manuellement les champs préremplis. Cette étape garantit que les informations extraites sont conformes à la réalité du ticket.

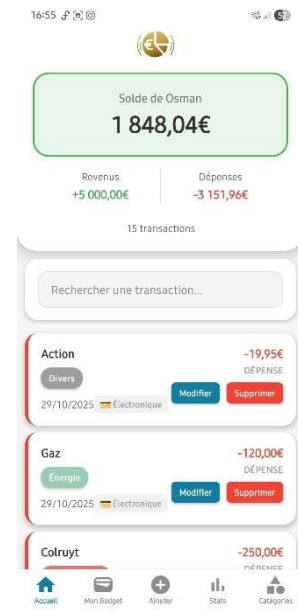
Défis rencontrés et solutions apportées : La reconnaissance de texte peut produire des erreurs selon la qualité de l'image (ticket flou, mal cadré, etc.). Pour éviter toute saisie incorrecte, le formulaire reste entièrement modifiable après extraction. L'utilisateur peut corriger les champs avant validation. Si une erreur passe inaperçue, la transaction peut être modifiée à tout moment via l'écran d'accueil. Ce choix technique renforce la fiabilité du système tout en laissant à l'utilisateur le contrôle final sur ses données.

⁶ Voici la discussion qui a eu lieu pour l'implémentation avec DeepSeek :
<https://chat.deepseek.com/share/7didevyzzqyuo5vzc7>

Affichage des transactions (Accueil)

Description : L'écran d'accueil présente l'ensemble des transactions enregistrées. Chaque opération affiche ses détails essentiels : titre, montant, date, mode de paiement et catégorie, avec un badge coloré pour faciliter la lecture. Des boutons permettent de modifier ou supprimer une transaction directement depuis la liste.

Défis rencontrés et solutions apportées : La synchronisation des couleurs entre les catégories et les badges visuels a été assurée via une requête dédiée. La suppression d'une transaction déclenche une mise à jour immédiate de la liste, sans rechargement manuel. Enfin, le thème jour/nuit adapte dynamiquement les couleurs pour garantir une lisibilité optimale sur tous les appareils.



Tri chronologique des transactions

Description : Les transactions sont affichées du plus récent au plus ancien, selon leur date réelle d'enregistrement.

Défis rencontrés et solutions apportées : Le tri par date a été mis en œuvre grâce à une logique personnalisée inspirée d'un exemple trouvé sur GeeksforGeeks⁷. Au lieu d'utiliser la date de soumission, le système trie les opérations selon leur champ date, garantissant une lecture fidèle à la chronologie des dépenses⁸ :

```
const sortedTransactions = data.sort((a, b) => {  
  | return new Date(b.date).getTime() - new Date(a.date).getTime();  
});
```

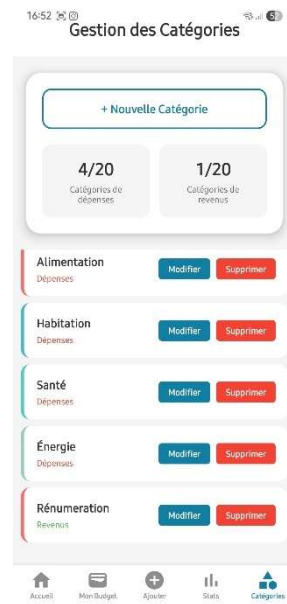
⁷ Référence du script mentionné : <https://www.geeksforgeeks.org/javascript/sort-an-object-array-by-date-in-javascript/>

⁸ Ce script se trouve dans le répertoire : « ..\app\tabs\home.tsx » L39

Gestion des Catégories personnalisées

Description : Les utilisateurs peuvent créer, modifier, supprimer et sélectionner des catégories pour organiser leurs transactions. Chaque catégorie est associée à une couleur unique, choisie automatiquement depuis une palette prédéfinie. La catégorie “Divers” est protégée et utilisée comme valeur par défaut pour les revenus et les dépenses. Ces couleurs sont également réutilisées dans les graphiques statistiques pour assurer une cohérence visuelle.

Défis rencontrés et solutions apportées : Pour garantir la lisibilité et éviter la surcharge visuelle, une limite stricte de 20 catégories par type (revenu ou dépense) a été imposée, hors “Divers”. Cette contrainte, ainsi que l’ensemble de la logique de gestion des catégories, est centralisée dans le fichier storage.ts



Initialisation de la base et insertion de “Divers” :

Lors du démarrage de l’application, la base de données est initialisée avec les tables nécessaires, et la catégorie “Divers” est insérée automatiquement si elle n’existe pas⁹ :

```
await db.runAsync(  
  `INSERT OR IGNORE INTO categories (name, color, type) VALUES (?, ?, ?)`,  
  ['Divers', '#9E9E9E', 'expense']  
);  
await db.runAsync(  
  `INSERT OR IGNORE INTO categories (name, color, type) VALUES (?, ?, ?)`,  
  ['Divers', '#9E9E9E', 'income']  
);
```

Attribution des couleurs :

Les couleurs sont sélectionnées à partir d’une palette fixe ¹⁰ :	Lors de l’ajout d’une catégorie, la couleur est choisie automatiquement parmi celles non utilisées, ou réattribuée de manière cyclique ¹¹ :
<pre>const CATEGORY_COLORS = ['#FF6B6B', '#4ECDC4', '#45B7D1', '#96CEB4', '#FFAA7', '#DDA0DD', '#98D8C8', '#F7DC6F', '#8B8FCE', '#85C1E9', '#F8C471', '#B2E0AA', '#F1948A', '#B5C1E9', '#D7BDE2', '#F9E79F', '#ABEBC6', '#ED8B99', '#AED6F1', '#FAD7A8'];</pre>	<pre>const usedColors = existingCategories.map(cat => cat.color); const availableColor = CATEGORY_COLORS.find(color => !usedColors.includes(color)) CATEGORY_COLORS[currentCount % CATEGORY_COLORS.length];</pre>

Ajout avec vérification de la limite :

Avant toute insertion, le système vérifie que le nombre de catégories personnalisées n’excède pas 20¹² :

```
if (currentCount >= 20) {  
  throw new Error(`Limite de 20 catégories ${type === 'income' ? 'de revenus' : 'de dépenses'} atteinte.`);  
}
```

⁹ Voir L45 de « ..\app\utils\storage.ts »

¹⁰ Voir L5 de « ..\app\utils\storage.ts »

¹¹ Voir L75 de « ..\app\utils\storage.ts »

¹² Voir L70 de « ..\app\utils\storage.ts »

Suppression et réaffectation :

La suppression de “Divers” est interdite et lorsqu’une autre catégorie est supprimée, toutes les transactions associées sont automatiquement réaffectées à “Divers”¹³ :

```
// Ne pas permettre la suppression de "Divers"
if (categoryName === 'Divers') {
  throw new Error('Impossible de supprimer la catégorie "Divers"');
}

// 1. Mettre à jour toutes les transactions avec cette catégorie vers "Divers"
await db.runAsync(
  "UPDATE transactions SET category = 'Divers' WHERE category = ? AND type = ?",
  [categoryName, type]
);

// 2. Supprimer la catégorie
await db.runAsync(
  "DELETE FROM categories WHERE name = ? AND type = ?",
  [categoryName, type]
);
```

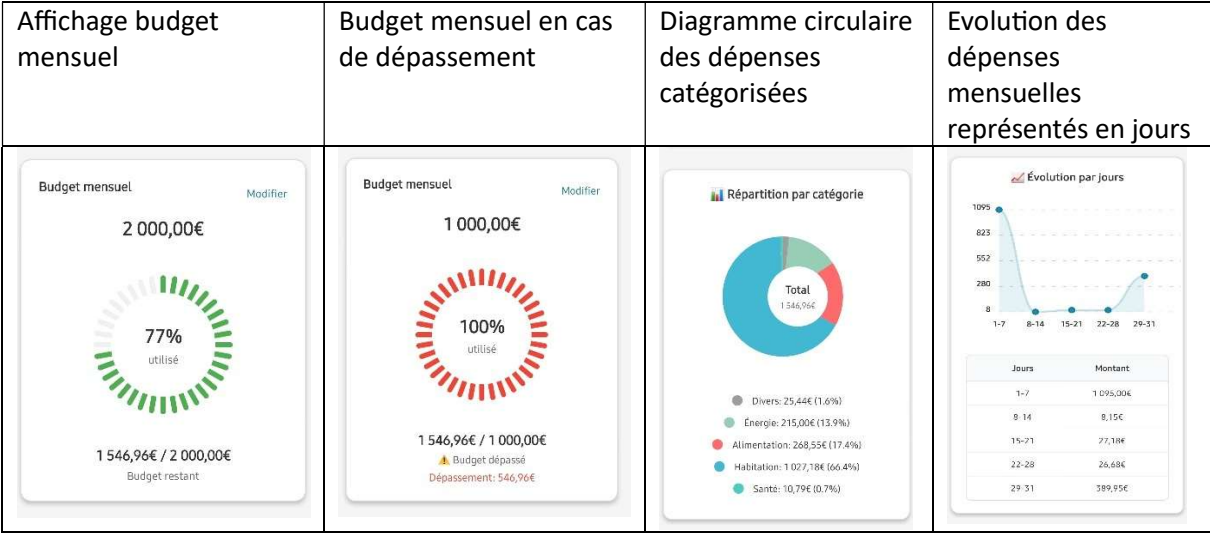
Cette organisation garantit une gestion robuste, cohérente et visuellement harmonieuse des catégories, tout en assurant la stabilité des données et une expérience utilisateur fluide.

Suivi mensuel et statistiques visuelles

Description : L’application permet aux utilisateurs de suivre leur budget mensuel et d’analyser leurs dépenses à travers des graphiques interactifs. Chaque mois affiche le solde, les revenus, les dépenses, ainsi qu’une barre de progression du budget utilisé. Des visualisations complémentaires présentent l’évolution hebdomadaire des dépenses, leur répartition par catégorie et par mode de paiement.

Défis rencontrés et solutions apportées : La saisie du budget a été sécurisée par une validation du format et une conversion automatique en valeur exploitable. Une alerte visuelle est déclenchée en cas de dépassement. Pour les statistiques, l’agrégation des données par semaine et par catégorie a nécessité des requêtes spécifiques en base SQLite. Les couleurs des catégories sont réutilisées dans les graphiques pour assurer une cohérence visuelle. Enfin, la navigation entre les mois a été optimisée pour garantir une continuité fluide dans le suivi des données.

¹³ Voir L260 de « ../app/utils/storage.ts »



Choix de la base de données locale (SQLite)

Description : Pour permettre l'enregistrement des revenus et des dépenses, l'application utilise **SQLite** comme moteur de base de données local. Ce choix permet d'effectuer des opérations CRUD (Create, Read, Update, Delete) directement sur l'appareil, sans dépendance réseau. SQLite offre une structure relationnelle idéale pour gérer les liens entre transactions et catégories, tout en garantissant des performances fiables en mode hors-ligne.

Défis rencontrés et solutions apportées : Lors de la conception, plusieurs options ont été envisagées pour le stockage des données. Une comparaison technique a été réalisée avec l'aide de l'IA DeepSeek¹⁴, mettant en balance les avantages et inconvénients de chaque solution :

- **WatermelonDB** offrait une excellente performance sur de gros volumes et une synchronisation automatique de l'interface, mais sa complexité et sa courbe d'apprentissage étaient jugées excessives pour un projet étudiant.
- **AsyncStorage**, bien que simple à implémenter, ne permettait pas de gérer des relations complexes ni d'effectuer des requêtes structurées.
- **SQLite** s'est imposé comme le meilleur compromis entre puissance, simplicité et compatibilité avec Expo.

Ce choix technique a permis de construire une base robuste, évolutive et parfaitement adaptée aux besoins de l'application, tout en restant accessible pour un développement rapide et maîtrisé.

¹⁴ Discussion avec l'IA DeepSeek : <https://chat.deepseek.com/share/206d6a3su1jpnh24de>

Analyse

L'application **Budgio** repose sur une architecture modulaire respectant les principes SOLID¹⁵, où chaque couche remplit un rôle spécifique, interconnecté pour assurer une expérience fluide. Cette analyse explore les choix techniques et les relations entre les composants qui assurent une expérience utilisateur fluide pour la gestion budgétaire.

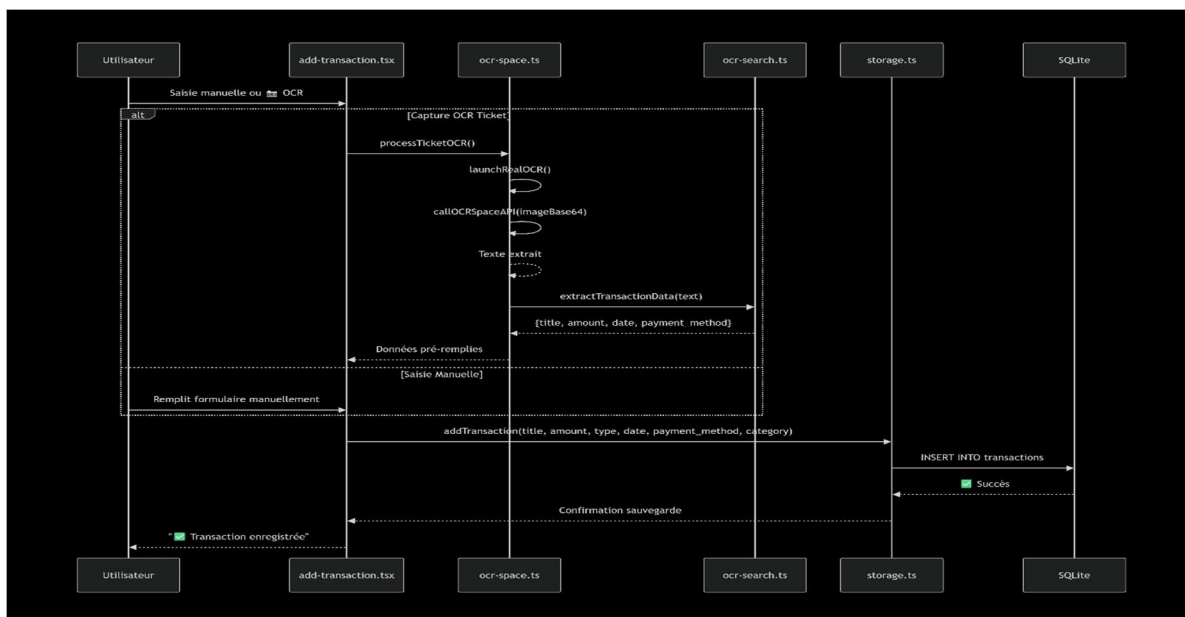
Architecture globale

L'application est organisée en quatre couches principales :

- I. **Couche présentation** : Gère l'interface utilisateur via des écrans React Native et transmet les actions utilisateur aux utilitaires sous-jacents.
- II. **Couche données** : Gère l'interface utilisateur via des écrans React Native et transmet les actions utilisateur aux utilitaires sous-jacents.
- III. **Services autonomes** : Automatisent des fonctionnalités avancées comme la reconnaissance OCR avec ocr-space.ts et ocr-search.ts.
- IV. **Utilitaires transversaux** : Regroupent des fonctions réutilisables pour la gestion des couleurs, thèmes et interactions utilisateur.

Interactions utilisateur : Cas concret de l'ajout de transaction

Lorsque César enregistre une dépense, l'interaction passe par `add-transaction.tsx`, qui valide les données localement avant de les transmettre à `storage.ts` via `addTransaction()`. Une requête SQL est alors exécutée dans SQLite, et les écrans `home.tsx`, `my-budget.tsx` et `stats.tsx` se mettent à jour via `useFocusEffect`. En alternative, César peut utiliser l'OCR : la capture est lancée via `ocr-speech.ts`, l'image envoyée à `OCR.space`, puis analysée par `ocr-search.ts` avec `extractTransactionData()`. Les données extraites sont renvoyées à `add-transaction.tsx`, qui pré-remplit le formulaire. Il ne reste plus qu'à valider. Le diagramme en annexe illustre ce flux :

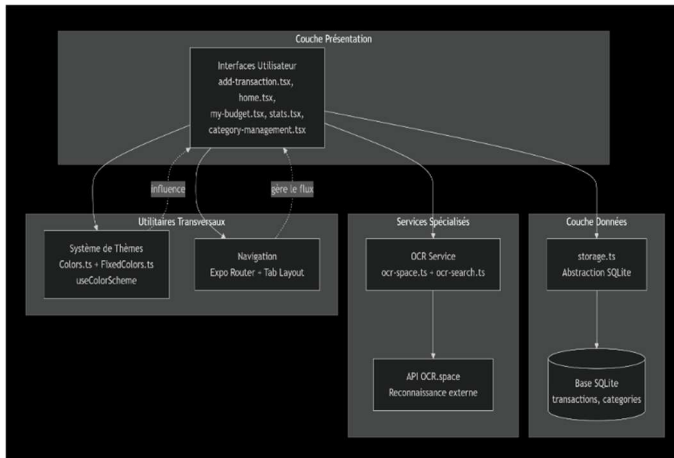


¹⁵ Principes SOLID visent à structurer un code clair, modulaire et flexible en séparant les responsabilités, facilitant l'extension, garantissant l'interopérabilité, adaptant les interfaces aux besoins et réduisant le couplage des dépendances. [SOLID : les 5 premiers principes de conception orientée objet | DigitalOcean](#)

Relations entre les couches de l'architecture

Les composants de l'application sont organisés en couches distinctes mais interdépendantes. Le diagramme ci-dessous illustre les relations entre les différentes couches de l'architecture :

L'architecture de Budgio repose sur une séparation claire des responsabilités organisée en quatre couches principales :



La **couche présentation** agit comme une interface pour les utilisateurs.

La **couche donnée** centralise les interactions avec la base SQLite

Les **services spécialisés** automatisent des processus complexes comme la reconnaissance optique de caractères (OCR)

Les **utilitaires transversaux** apportent modularité et réutilisabilité

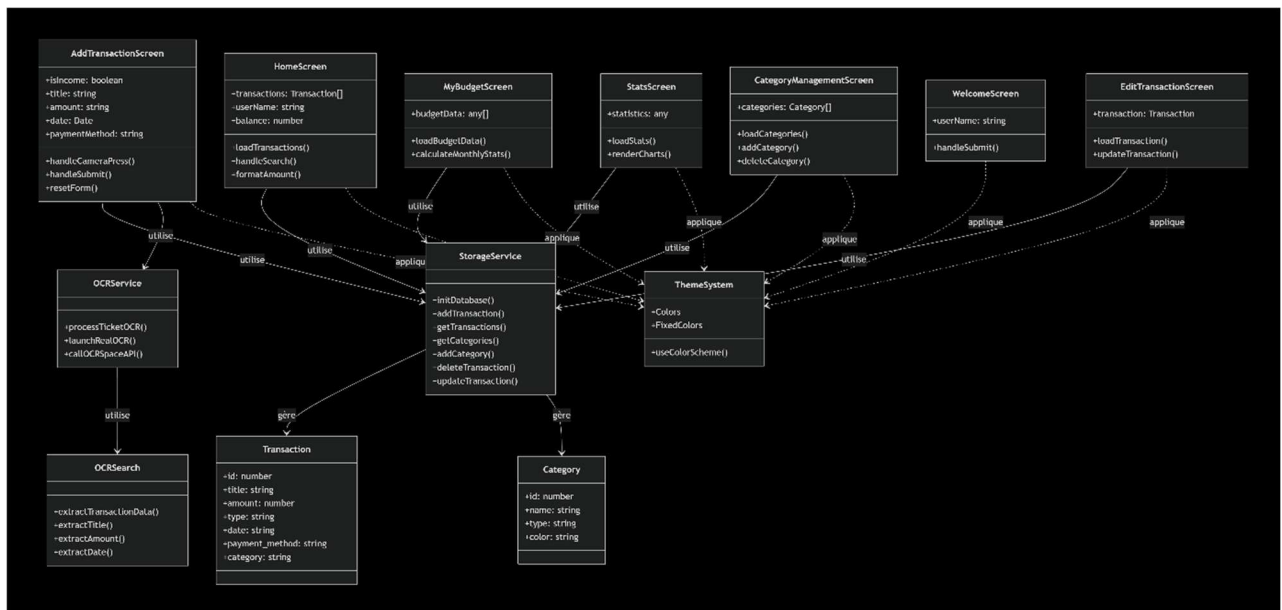
Ces interactions mettent en évidence la coopération entre les différentes couches qui composent l'architecture de l'application. Par exemple, `add-transaction.tsx` intercepte les actions utilisateur (saisie manuelle ou capture OCR) et transmet les données à `storage.ts`, qui assure leur persistance. Les écrans `home.tsx`, `my-budget.tsx` et `stats.tsx` se synchronisent automatiquement via `useFocusEffect` pour refléter les changements en temps réel. Du côté des services, `ocr-space.ts` orchestre la capture d'image et délègue l'analyse textuelle à `ocr-search.ts`, qui collabore avec l'API `OCR.space` pour structurer les données extraites. Enfin, les utilitaires garantissent une cohérence visuelle entre les écrans et une navigation fluide entre les sections de l'application.

Cette interconnexion garantit une gestion robuste des processus métier et une expérience utilisateur intuitive, réactive et cohérente.

Vue d'ensemble avec le diagramme de classes principal

Le diagramme global ci-dessous détaille les relations entre toutes les classes principales de **Budgio**, depuis l'interface utilisateur jusqu'à la gestion des données. Ce schéma met en avant l'organisation modulaire et l'interconnexion des différentes parties de l'architecture.

Voici un diagramme global montrant la structure complète de l'application et les relations entre ses composants :



L'analyse révèle la robustesse et la modularité de l'architecture de Budgio. L'interconnexion efficace de ses 7 écrans principaux avec les services centralisés permet une expérience utilisateur fluide et cohérente. La séparation claire entre la couche présentation (écrans), la couche service (OCR, Stockage) et la couche données (Transactions, Catégories) garantit une maintenance simplifiée et une évolutivité future.

Chaque écran spécialisé

(HomeScreen, AddTransactionScreen, MyBudgetScreen, StatsScreen, CategoryManagementScreen, EditTransactionScreen, WelcomeScreen) interagit de manière standardisée avec le StorageService central, tandis que le système de thèmes unifié assure une cohérence visuelle sur l'ensemble de l'application.

Cette architecture solide, avec ses composants spécialisés et réutilisables, positionne **Budgio** comme une solution robuste et évolutive pour la gestion budgétaire mobile, capable de s'adapter facilement à de nouvelles fonctionnalités.

UX/UI

ImageButtons pour une interface intuitive

Afin d'améliorer l'expérience utilisateur et de rendre l'interface plus intuitive, des **ImageButtons** ont été intégrés à l'application. Ces boutons visuels permettent une navigation plus fluide et une reconnaissance immédiate des actions disponibles, en s'appuyant sur des icônes familières et cohérentes.

Les icônes intégrées proviennent de la bibliothèque Expo Vector Icons¹⁶, reconnue pour sa diversité et sa compatibilité avec les standards actuels du design d'interface. Leur utilisation permet de simplifier la lecture des fonctionnalités, d'alléger la charge cognitive de l'utilisateur et de favoriser une interaction plus intuitive, en particulier sur les appareils

¹⁶ Voici la référence des icônes : <https://icons.expo.fyi/Index>

mobiles. Ce choix graphique contribue à une interface à la fois accessible et esthétique, en parfaite adéquation avec les principes fondamentaux du design UX/UI.

Ajout d'une barre de recherche (SearchBar)

Une barre de recherche a été intégrée juste en dessous de la section du solde afin de permettre aux utilisateurs de filtrer rapidement les transactions affichées. La recherche s'effectue par **titre** ou **catégorie**, ce qui améliore considérablement la navigation, notamment lorsque la liste des transactions devient volumineuse. Cette fonctionnalité renforce l'efficacité de l'interface et répond aux besoins d'accessibilité et de rapidité.

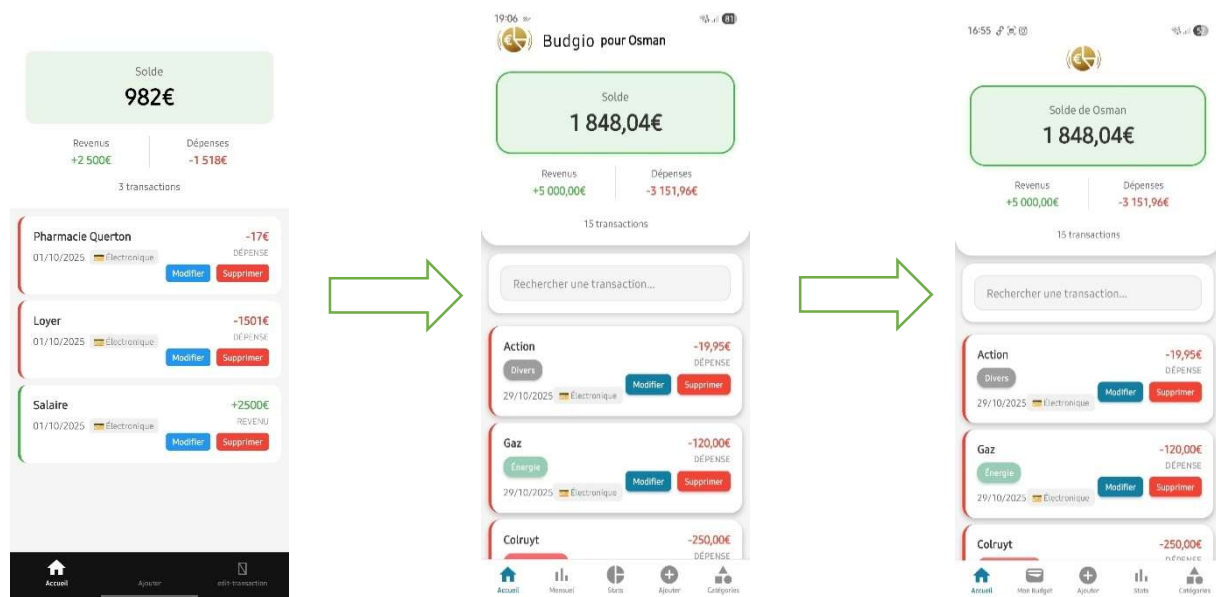
Design : Une interface inspirée de l'univers des cartes

L'interface adopte un design épuré mettant en valeur un fond en forme de carte, sur lequel les données sont présentées dans un rectangle aux coins arrondis. Ce choix esthétique, qui évoque la silhouette familière d'une carte bancaire, confère à l'application une identité visuelle moderne et raffinée. En plus de renforcer la lisibilité des informations, cette approche crée une continuité graphique harmonieuse, tout en apportant une touche de sophistication à l'ensemble de l'expérience utilisateur.

Attribution de couleurs par catégorie

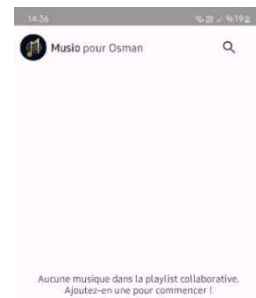
Chaque transaction, qu'il s'agisse d'un revenu ou d'une dépense, est associée à une catégorie distincte identifiée par une couleur spécifique. Ce système de codage visuel facilite une lecture fluide et intuitive des données, tout en permettant un repérage immédiat des types de flux financiers. Il contribue également à améliorer le confort visuel et l'expérience globale de l'utilisateur. Ces catégories colorées sont réutilisées dans un diagramme circulaire qui offre une représentation synthétique et visuellement cohérente de la répartition des transactions. L'utilisateur bénéficie ainsi d'une vue d'ensemble claire et rapide de ses habitudes financières, renforçant sa capacité à analyser et à gérer son budget efficacement.

Et voici l'évolution de la page home.tsx comme mentionné :



Héritage visuel – Une signature graphique entre Musio et Budgio

L'application **Budgio** conserve une trace subtile de son prédécesseur **Musio**¹⁷ à travers un élément graphique emblématique : l'en-tête personnalisé. Initialement formulé sous la forme « *Musio pour nomUtilisateur* », ce détail est repris dans Budgio avec « *Budgio pour nomUtilisateur* », avant d'évoluer vers « *Solde de nomUtilisateur* ». Ce choix n'est pas anodin : il incarne une **signature visuelle** propre au développeur, une marque d'identité qui traverse les projets. Déjà apprécié dans Musio pour sa **clarté** et son **caractère accueillant**, cet en-tête personnalisé renforce la cohérence entre les interfaces tout en créant un lien affectif avec l'utilisateur. Il s'agit d'un clin d'œil discret mais significatif, qui témoigne d'une volonté de continuité, de reconnaissance et d'attention portée à l'expérience utilisateur.



Mode jour/nuit automatique

L'application intègre un **mode jour/nuit** qui s'adapte automatiquement au mode activé sur l'appareil mobile. Cette fonctionnalité **améliore le confort visuel** des utilisateurs en ajustant les couleurs et l'apparence de l'interface selon les conditions d'éclairage. Le mode jour offre une meilleure visibilité en lumière vive, tandis que le mode nuit réduit la fatigue oculaire dans des environnements peu éclairés.

Remplacement de l'activité MobileXpo

L'évènement **MobileXpo** n'ayant pas été organisé cette année, une alternative a été mise en place afin d'obtenir un retour utilisateur sur le projet. Une étudiante en deuxième année, passionnée et curieuse, a été sollicitée pour tester l'application. Son profil correspondait parfaitement au **public ciblé** dans le cadre du **MobileXpo**. Elle avait déjà fourni un retour particulièrement pertinent sur le projet Mobile Musio, ce qui justifiait pleinement son implication. Ses retours ont permis de valider certains choix, notamment sur l'ergonomie et la clarté des fonctionnalités.

Afin de structurer ce retour de manière claire, les impressions de L'étudiante ont été regroupées selon les aspects positifs, les points à améliorer et ses préférences en matière d'interface.

Feedback de l'étudiante

Points positifs : Le test de l'application par l'étudiante a permis de mettre en évidence plusieurs éléments appréciés. L'affichage personnalisé « *solde de + prénom* » a été perçu comme une touche **accueillante** et **valorisante**. Ce choix s'inscrit dans la continuité du projet *Musio*, où une personnalisation similaire « *Musio pour elle* » avait déjà été remarquée positivement. Reprendre cette approche dans le projet actuel a contribué à renforcer le sentiment d'attention portée à l'utilisateur.

Elle a également salué le **design** en style *carte bancaire*, qu'elle a trouvé à la fois **élégant** et parfaitement **adapté au thème** de la gestion budgétaire. Ce choix graphique contribue à une

¹⁷ Musio est l'application qui a dû être développée en Android java pour le cours de mobile 1 (2024-2025)

interface cohérente et agréable à utiliser. Par ailleurs, la représentation statistique des dépenses a été bien accueillie. Elle a souligné que cette visualisation permettait de mieux comprendre ses habitudes de consommation. Enfin, la fonctionnalité de budget mensuel, accompagnée d'un indicateur visuel du pourcentage consommé, a été jugée **claire et pertinente, facilitant le suivi des dépenses**.

Point discuté : L'étudiante a exprimé une réserve concernant la possibilité de modifier une transaction après son enregistrement, notamment lorsqu'elle provient d'un scan OCR. Selon elle, cette fonctionnalité **semble peu utile et pourrait être évitée**. Toutefois, ce choix a été maintenu dans le projet, car il offre une flexibilité appréciable : permettre à l'utilisateur de corriger une erreur ou un oubli sans devoir supprimer puis recréer la transaction. Cette option vise à améliorer l'expérience globale en cas de saisie imprécise.

Bug détecté : Lors de la démonstration, un **crash critique** s'est produit au moment de la saisie du nom utilisateur. L'origine du problème a été rapidement identifiée : une **erreur de répertoire dans le chemin de redirection vers la page d'accueil**¹⁸, spécifique aux nouveaux utilisateurs. Cette anomalie provoquait l'arrêt brutal de l'application sur certains appareils.

Voici l'erreur	Voici la correction
<pre>try { // Sauvegarder le nom d'utilisateur await AsyncStorage.setItem('userName', userName.trim()); // Rediriger vers l'écran principal router.replace('/home'); }</pre>	<pre>try { // Sauvegarder le nom d'utilisateur await AsyncStorage.setItem('userName', userName.trim()); // Rediriger vers l'écran principal router.replace('/(tabs)/home'); }</pre>

Ce test s'est révélé particulièrement utile, car il a été effectué sur un appareil différent de celui utilisé en phase de développement, permettant ainsi de détecter un dysfonctionnement qui aurait pu passer inaperçu. La correction rapide de ce bug a permis d'éviter une pénalité lors de l'évaluation et d'améliorer la robustesse globale de l'application.

Préférence d'interface : Interrogée sur l'organisation des sections *budget* et *statistiques*, l'étudiante a recommandé de les maintenir séparées. Selon elle, il est préférable de ne pas surcharger une page afin de préserver la lisibilité et la fluidité de navigation. Cette remarque a conforté le choix initial d'une interface segmentée, pensée pour offrir une expérience utilisateur claire et structurée.

Point négatif : L'étudiante a indiqué qu'il était dommage que l'application ne propose pas de fonctionnalité de type « *tirelire* ». Elle aurait aimé pouvoir ajouter des montants dans un espace dédié afin d'atteindre un objectif précis, comme un projet personnel ou un achat important. Elle a également suggéré l'ajout d'une réserve d'urgence, permettant de mettre de côté une somme en cas d'imprévu, tel qu'un accident ou une maladie. Selon elle, ces options renforceraient la dimension préventive et proactive de la gestion financière.

Dans l'ensemble, l'étudiante a indiqué qu'elle n'avait rien à redire sur l'application et qu'elle trouvait l'expérience utilisateur agréable, fluide et bien conçue.

¹⁸ Ce script se trouve dans le répertoire : « ..\app\welcome-screen.tsx »

Limitations et développement futur

Limitations

Connexion Internet

L'application fonctionne entièrement hors ligne, à **l'exception de la fonctionnalité d'OCR** qui **nécessite une connexion Internet** pour analyser les tickets de caisse. En l'absence de réseau, seule cette fonctionnalité devient indisponible, tandis que toutes les autres restent pleinement opérationnelles.

Stockage local des données

Toutes les transactions sont **enregistrées localement** sur l'appareil. Il n'existe pas de mécanisme de sauvegarde ni de synchronisation multi-appareil, ce qui rend les données vulnérables en cas de perte ou de changement de terminal.

Limitation de l'API OCR gratuite

L'application utilise la version gratuite d'**OCR Space**, qui impose une limite de **25 000 requêtes par mois**. En cas de dépassement, la reconnaissance de texte devient temporairement indisponible, ce qui peut affecter l'expérience utilisateur en période de forte activité.

Fiabilité de l'OCR

La reconnaissance de texte fonctionne correctement dans des conditions standards, mais reste **sensible à la qualité des images**. Les tickets flous ou mal cadrés peuvent entraîner des erreurs d'extraction, obligeant l'utilisateur à corriger manuellement les champs.

Limite de catégories personnalisées

Le nombre de **catégories est limité à 20** par type (revenu ou dépense). Cette contrainte, bien qu'utile pour préserver la lisibilité, peut restreindre la personnalisation pour les utilisateurs ayant des besoins de classification plus complexes.

Statistiques limitées

Les graphiques proposés se concentrent sur une vue **mensuelle**. Il n'est pas encore possible d'explorer des tendances sur le long terme ni d'appliquer des filtres avancés (par titre, mode de paiement, etc.).

Usage mono-utilisateur

L'application est conçue pour un **usage individuel**. Il n'existe pas de système de comptes ni de partage des données, ce qui limite les usages collaboratifs ou familiaux.

Gestion des devises

L'application est actuellement **limitée à l'euro**, sans possibilité de choisir une autre devise ni d'effectuer des conversions. Le format monétaire suit les conventions françaises, ce qui peut nuire à la lisibilité pour un public international.

Développements futurs

Mise en place d'une fonctionnalité de tirelire :

À la suite du retour de l'étudiante, une section dédiée à la **gestion d'objectifs d'épargne** sera envisagée dans l'application. Cette fonctionnalité permettra à l'utilisateur de créer des **tirelres virtuelles** dans lesquelles il pourra ajouter des montants progressivement, en vue d'atteindre un **objectif spécifique** ou de constituer une **réserve d'urgence**. Un bouton « *Ajouter à ma tirelire* » pourrait être intégré, offrant à l'utilisateur la possibilité de suivre l'évolution de ses économies et de renforcer sa motivation. Les tirelres pourront être personnalisées selon le type d'objectif (voyage, achat, imprévu) et s'appuieront sur une logique simple de cumul, tout en respectant les contraintes techniques actuelles.

Amélioration de la reconnaissance OCR

L'intégration d'une technologie OCR plus performante ou le recours à une **API d'intelligence artificielle spécialisée** permettrait de réduire les erreurs d'extraction, même dans des conditions d'image imparfaites, et de limiter les corrections manuelles.

Sauvegarde et synchronisation des données

La mise en place d'un système de **sauvegarde cloud** et de **synchronisation multi-appareil** garantirait la sécurité des données et leur accessibilité depuis différents terminaux.

Extension des statistiques

Le module de visualisation pourrait être enrichi avec des **filtres avancés** (par catégorie, mode de paiement, période personnalisée) et des **analyses longitudinales** pour mieux suivre les tendances financières.

Catégorisation intelligente

L'intégration d'un **moteur de suggestion basé sur l'IA** pourrait automatiser la classification des transactions selon leur libellé, réduisant ainsi la charge cognitive de l'utilisateur.

Intégration de données économiques en temps réel

L'ajout d'**API externes** permettrait d'afficher les **taux de change** (euro, dollar, etc.), les **prix des biens courants** (essence, alimentation, etc.) et des **indicateurs économiques** (inflation, panier moyen), offrant ainsi une lecture plus contextuelle et dynamique des dépenses.

Gestion multidevise

Une évolution majeure consisterait à **lever le verrouillage sur l'euro** en permettant à l'utilisateur de choisir sa devise principale. L'intégration d'un **système de conversion automatique et d'affichage des taux de change en temps réel** rendrait l'application plus adaptée à un usage international. Le formatage des montants serait également ajusté selon les conventions locales, améliorant la lisibilité et l'accessibilité pour un public plus large.

Ces évolutions permettront non seulement d'améliorer l'expérience utilisateur, mais également de renforcer l'attractivité et la pérennité de l'application **Budgio**.

Conclusion

Ce projet m'a offert une liberté créative totale, ce qui a rendu l'expérience à la fois motivante et profondément enrichissante. Pouvoir concevoir une application selon mes propres idées, sans contrainte imposée, m'a permis de m'investir pleinement, de tester mes limites et de renforcer mes compétences techniques. J'avais déjà ressenti cet élan personnel lors du projet Mobile I avec Musio, et Budgio m'a procuré le même sentiment d'accomplissement. Transformer une idée en une application fonctionnelle, pensée pour un usage concret, a été une aventure stimulante.

Concernant l'intelligence artificielle, mes précédents travaux m'avaient amené à exprimer certaines réserves, notamment sur sa fiabilité et la nécessité de garder un regard critique. Ces précautions restent valables, mais dans **Budgio**, l'IA s'est révélée être un véritable levier. Elle m'a permis de surmonter des blocages techniques, comme le choix d'une API OCR adaptée ou la résolution du problème des autorisations par défaut. Là où la documentation était insuffisante, elle m'a guidé avec pertinence et efficacité. Cette fois, je pense avoir su l'utiliser au bon moment et au bon endroit.

Le retour de l'étudiante, qui correspondait parfaitement au public cible du MobileXpo, a été précieux. Elle a validé plusieurs choix ergonomiques et graphiques, comme la clarté des statistiques, le design inspiré des cartes bancaires et la personnalisation du solde. Elle a également proposé une idée pertinente : intégrer une fonctionnalité de type tirelire, permettant à l'utilisateur de mettre de côté des montants pour atteindre un objectif ou constituer une réserve d'urgence. Bien que cette option n'ait pas été mise en place dans cette version, elle représente une piste intéressante pour la suite.

Si je devais améliorer ce projet, je me concentrerais sur trois axes principaux :

- ✓ L'ajout d'une **fonctionnalité de tirelire**, pour permettre à l'utilisateur de gérer des objectifs d'épargne ou des réserves en cas d'imprévu.
- ✓ L'**amélioration de la fiabilité de l'OCR**, en intégrant une technologie plus robuste capable de mieux gérer les images imparfaites et de limiter les corrections manuelles.
- ✓ La **synchronisation multi-appareil**, via un système de sauvegarde cloud, afin de garantir la sécurité des données et une expérience fluide sur différents terminaux.

Bien que l'évaluation académique soit un aspect du projet, je retiens surtout l'opportunité unique de concrétiser une idée personnelle dans un cadre d'apprentissage. Ce projet m'a permis de progresser techniquement, de résoudre des problèmes concrets et de vivre une expérience complète, de l'idée à la réalisation. **Budgio** est le fruit d'un parcours riche en exploration, en remises en question et en réussites. Je suis fier d'avoir pu lui donner vie, et je considère cette étape comme un jalon important dans mon évolution personnelle et professionnelle. Elle me donne confiance pour relever des défis encore plus ambitieux à l'avenir.

Liens YouTube vers les vidéos

Vidéo de présentation

<https://youtu.be/cdfPwDit-NA>

Vidéo Bonus

<https://youtu.be/UXMsGvUT5n0>